MATH / PRINT EDITION

Calculus for AI, from scratch

Differentiation from first principles up to gradients, Jacobians and Hessians, with every idea tied to the place it shows up in a neural network.

Govinda Vurjana · https://govinda-vurjana.github.io/me/math/calculus-for-ai/ · Oct 2026

Every neural network learns the same way. It makes a prediction, measures how wrong it was, and adjusts each of its weights in whichever direction makes it less wrong. Finding that direction is calculus. This series builds that calculus from nothing: school algebra is enough to start, and nothing is asked to be memorised without first seeing where it comes from.

A model learning, by hand

Here is the whole idea at its smallest. A model with one weight w predicts $\hat{y} = wx$. Show it one example, $x = 2$, whose right answer is $y = 6$, and score its error with the squared loss:

$$L(w) = (wx - y)^2 = (2w - 6)^2$$

Its derivative (Part 3 shows how to get it) is

$$\frac{dL}{dw} = 2x\,(wx - y) = 4\,(2w - 6)$$

Start from a bad guess, $w = 1$, and repeat one rule: move w a small step against the derivative, $w \leftarrow w - 0.05\,\frac{dL}{dw}$.

STEP	w	PREDICTION $2w$	LOSS	dL/dw
0	1.00	2.00	16.00	-16.00
1	1.80	3.60	5.76	-9.60
2	2.28	4.56	2.07	-5.76
3	2.57	5.14	0.75	-3.46
4	2.74	5.48	0.27	-2.07

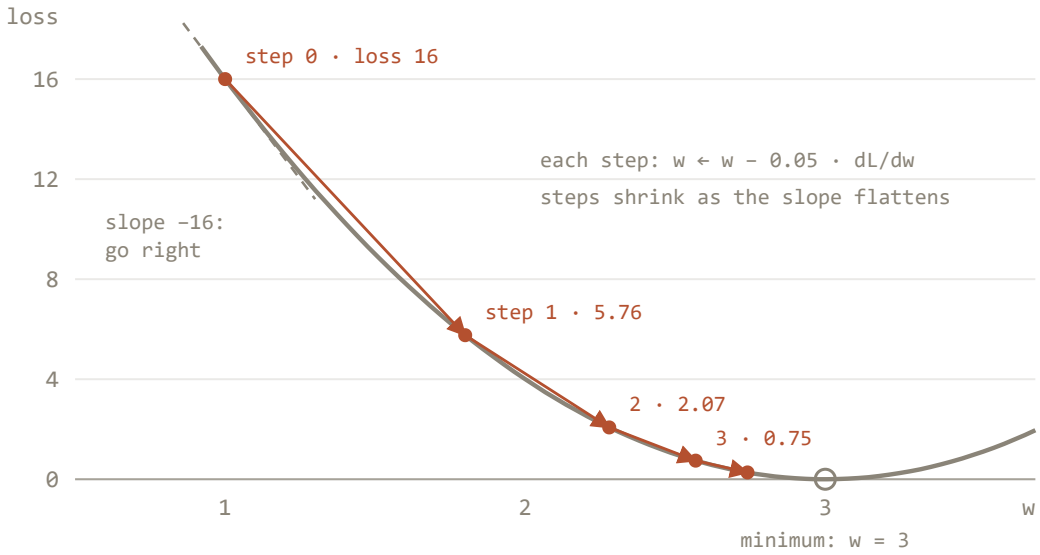


Figure 1. The same steps on the loss curve. Each one moves against the slope, and because the slope flattens near the bottom, the steps shrink on their own.

The derivative is negative, which says "increasing w lowers the loss", so w goes up. As the error shrinks the derivative shrinks with it, and the steps get shorter on their own. The weight closes in on $w = 3$, where the prediction is exactly right.

A large language model runs this same loop over billions of weights at once. The optimiser is more refined and the arithmetic is enormous, but every step still needs the derivative of the loss with respect to every weight, and the chain rule is what supplies it.

Read in order

1. What a derivative measures. Rate of change, built from first principles.
2. The derivative rules, and why they work. Powers, sums, products and quotients.
3. The chain rule. Functions inside functions, and backpropagation in miniature.
4. Exponentials, logarithms and sines. The functions behind sigmoid, softmax, cross-entropy and positional encodings.
5. Implicit differentiation. When y is tangled up with x.
6. The second derivative. Curvature, and why it caps the learning rate.
7. Partial derivatives and one neuron. Many inputs, and a first backward pass by hand.
8. The gradient, and which way is downhill. Gradients, directional derivatives and gradient descent.
9. Jacobians and Hessians. Derivatives of whole layers, and the shape of the loss landscape.

Then read The hidden meaning of the calculus, which reads every result in the series as a statement about how a network learns.

How each part works

Each part runs in the same order: the intuition first, then the definition, a derivation, worked examples, where the idea shows up in AI, and exercises. Each part also has a *hidden meaning*: what the result says about how a network learns. Anyone can learn to compute a derivative; knowing what it means is what makes it useful for AI. On the site the answers are folded away, so you can try first. Wherever possible a result is checked with plain numbers as well as algebra.

Part 1: What a derivative measures

A derivative is a rate of change at a single point. Here it is built from first principles and tied to the one question training asks of every weight.

A function is a machine

A function takes an input and produces an output. Squaring is one:

$$f(x) = x^2$$

so $f(2) = 4$, $f(3) = 9$ and $f(10) = 100$. Think of it as a machine: x goes in, $f(x)$ comes out. Almost everything in AI is a function in this sense. An image goes in and a label comes out; a sentence goes in and a next word comes out. A neural network is one very large function, built by plugging many small ones together.

How fast is it changing?

Watch $f(x) = x^2$ as x climbs one step at a time:

x	$f(x)$	RISE FROM THE ROW ABOVE
1	1	
2	4	3
3	9	5
4	16	7

Each step of 1 in x produces a bigger rise than the last. The curve keeps getting steeper, so there is no single "slope of x^2": the slope depends on where you are. The **derivative** is the slope at one point, how fast the output is changing at that exact input.

A rate is always change in output divided by change in input. Drive 100 km in 2 hours and your average speed is $100/2 = 50$ km per hour. The derivative is the same idea.

The definition, built up

Start at x and move a small distance h, to $x + h$. The input changed by h, and the output changed by $f(x + h) - f(x)$. Their ratio is the average rate of change over that stretch:

$$\frac{f(x+h) - f(x)}{h}$$

You divide by h because a rate is *per unit of input*, just as speed is per hour. Geometrically, this is the slope of the straight line through two points on the curve, called a secant.

Now let h shrink. The second point slides toward the first, and the secant swings round until it just touches the curve at x. That touching line is the tangent, and its slope is the derivative:

$$f'(x) = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$

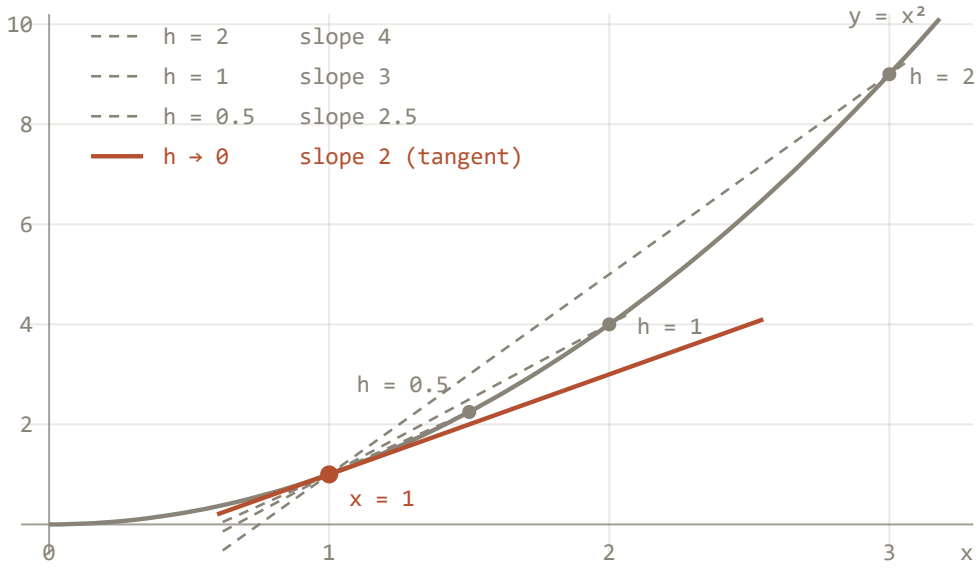


Figure 1. Secants of y = x² through x = 1. For h = 2, 1 and 0.5 their slopes are 4, 3 and 2.5. As h shrinks they turn into the tangent, whose slope is 2.

Why not simply set $h = 0$? Because then the fraction is $0/0$, which tells you nothing. The limit asks a different question: what number does the ratio *approach* as h gets very small? For a smooth curve that number exists, and it is the derivative.

Deriving the slope of x^2

Rather than memorise the answer, derive it. Put $f(x) = x^2$ into the definition:

$$f'(x) = \lim_{h \to 0} \frac{(x+h)^2 - x^2}{h}$$

Expand $(x+h)^2 = x^2 + 2xh + h^2$, and the x^2 terms cancel:

$$
\begin{aligned}
f'(x) &= \lim_{h \to 0} \frac{2xh + h^2}{h} \\
&= \lim_{h \to 0} \frac{h\,(2x + h)}{h} \\
&= \lim_{h \to 0} (2x + h)
\end{aligned}
$$

As h goes to zero, only $2x$ is left:

$$
\boxed{\frac{d}{dx}\, x^2 = 2x}
$$

This matches the figure. At $x = 1$ the secant slopes are $2 + h$: that is 4, 3 and 2.5 for the three secants, closing in on the tangent slope of 2.

Checking it with numbers

At $x = 3$ the formula says the slope is 6. Nudge x by 0.01:

$$
3.01^2 = 9.0601
$$

The output rose by 0.0601, almost exactly 6×0.01. That is what a derivative is in practice: a local exchange rate. Near $x = 3$, each small bit of input buys six times as much output.

$$
\Delta f \approx f'(x)\,\Delta x
$$

Keep this line in mind. Every use of derivatives in AI comes back to it.

What the number tells you

- **Sign.** If $f'(x) > 0$ the function is rising at x. If $f'(x) < 0$ it is falling. If $f'(x) = 0$ it is momentarily flat, as at the bottom of a valley or the top of a hill.
- **Size.** A large derivative means the output is very sensitive to the input there. A derivative near zero means small changes in the input barely matter.

Where this shows up in AI

A model's error, its *loss*, is a function of its weights. Write it $L(w)$. The derivative $\dfrac{dL}{dw}$ answers the one question training needs answered: *if I nudge this weight, does the error go up or down, and how fast?*

- $\dfrac{dL}{dw} = -16$: increasing w lowers the loss, steeply. Increase it.
- $\dfrac{dL}{dw} = 0.5$: increasing w raises the loss a little. Decrease it a little.

- $\frac{dL}{dw} \approx 0$: the loss is flat in this weight, so moving it barely helps right now.

Training is nothing more than following that answer, a small step at a time, for every weight. The topic page shows a one-weight model learning this way, and Part 8 does it for many weights at once.

> **HIDDEN MEANING**
>
> A derivative is a measure of responsibility. $\partial L/\partial w$ says how much of the current error is this weight's doing, and in which direction, so training is the work of assigning blame and correcting it. The answer is also only *local*: it treats the loss as a straight line near the current w, and a little further away that picture can be wrong. That is why a network learns by many small corrections rather than one leap to the answer.

Try it

1. Use the definition to find the derivative of $f(x) = 3x$.
2. Use the definition to find the derivative of $f(x) = x^3$. You will need $(x+h)^3 = x^3 + 3x^2h + 3xh^2 + h^3$.
3. Without any formula, estimate the slope of x^2 at $x = 5$ by computing $\frac{f(5.001) - f(5)}{0.001}$. How close is it to 2×5?

> **▼ ANSWERS**
>
> 1. $\frac{3(x+h) - 3x}{h} = \frac{3h}{h} = 3$. A straight line has the same slope everywhere.
> 2. $\frac{3x^2h + 3xh^2 + h^3}{h} = 3x^2 + 3xh + h^2$, which tends to $3x^2$.
> 3. $5.001^2 = 25.010001$, so the ratio is $0.010001/0.001 = 10.001$. The extra 0.001 is the leftover h in $2x + h$.

Part 2: The derivative rules, and why they work

A handful of rules cover most of what you need to differentiate by hand. Each one has a reason, and the reasons are easier to keep than the formulas.

Working from the definition every time would be slow. A few rules do the same job in a line, and each one follows from the definition in Part 1.

Constants, multiples and sums

$$\frac{d}{dx}\,c = 0$$

$$\frac{d}{dx}\big[a\,f(x)\big] = a\,f'(x)$$

$$\frac{d}{dx}\big[f(x) \pm g(x)\big] = f'(x) \pm g'(x)$$

- A constant never changes, so its rate of change is zero. Its graph is a flat line.
- Stretching a graph vertically by a stretches every slope by a.
- When two quantities are added, their rates add. Two taps filling one tank fill it at the sum of their rates.

The power rule

$$\frac{d}{dx}\,x^n = n\,x^{n-1}$$

You saw $n = 2$ in Part 1. For any whole number n, expanding $(x+h)^n$ gives $x^n + nx^{n-1}h$ plus terms containing h^2 or higher powers of h. Subtract x^n, divide by h, let $h \to 0$, and only nx^{n-1} survives.

The rule also works for negative and fractional powers, which makes it more useful than it looks. Rewrite fractions and roots as powers first:

$$\begin{aligned}\frac{d}{dx}\,\frac{1}{x^2} &= \frac{d}{dx}\,x^{-2} \\ &= -2x^{-3} = -\frac{2}{x^3}\end{aligned}$$

$$\frac{d}{dx}\sqrt{x} = \frac{d}{dx}\,x^{1/2}$$
$$= \tfrac{1}{2}\,x^{-1/2} = \frac{1}{2\sqrt{x}}$$

With the first three rules, a polynomial is differentiated term by term:

$$\frac{d}{dx}\left(8x^7 - 4x^3 + 9\right) = 56x^6 - 12x^2$$

The product rule

$$(fg)' = f'g + fg'$$

When two functions are multiplied, both are changing at once. Picture $f \cdot g$ as the area of a rectangle with sides f and g. Nudge x, and the width grows by Δf while the height grows by Δg.

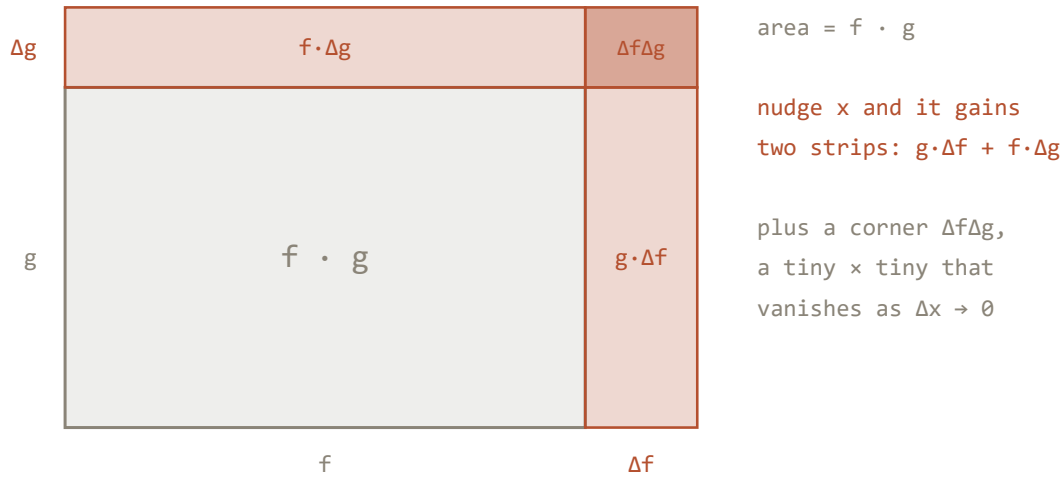


Figure 1. The product f·g as an area. When x moves, the area grows by two strips, g·Δf and f·Δg, plus a corner ΔfΔg.

The new area gains a strip $g\,\Delta f$ down one side, a strip $f\,\Delta g$ along the top, and a small corner $\Delta f\,\Delta g$. Divide by Δx and let it shrink. The strips become $f'g$ and fg', and the corner, a tiny quantity times another tiny quantity, disappears. That is the whole rule. It also shows why the answer is not simply $f'g'$: each factor's change is scaled by the *size* of the other factor.

Example: $y = x^2(x^3 + 1)$, with $f = x^2$ and $g = x^3 + 1$:

$$y' = (2x)(x^3 + 1) + (x^2)(3x^2)$$
$$= 5x^4 + 2x$$

Check by multiplying out first: $y = x^5 + x^2$, so $y' = 5x^4 + 2x$. Same answer.

The quotient rule

$$\left(\frac{f}{g}\right)' = \frac{f'g - fg'}{g^2}$$

In words: *bottom times derivative of top, minus top times derivative of bottom, all over bottom squared*. The minus sign has a reason. Raising the top raises the fraction; raising the bottom lowers it.

Example:

$$\begin{aligned}\frac{d}{dx}\,\frac{x^2}{x+1} &= \frac{(2x)(x+1) - (x^2)(1)}{(x+1)^2} \\ &= \frac{x^2 + 2x}{(x+1)^2}\end{aligned}$$

The quotient rule isn't really new. Writing f/g as $f \cdot g^{-1}$ turns it into the product rule and the power rule, joined by the chain rule from Part 3.

Where this shows up in AI

Loss functions are assembled from exactly these pieces.

- The squared error $(\hat{y} - y)^2$ is a power. A weight penalty such as λw^2 is a constant times a power.
- The loss on a batch of examples is the average of the loss on each one, so its derivative is the average of their derivatives. That is why a network can learn from a whole batch at once: the batch's gradient is just the average of each example's gradient.
- Products appear wherever two quantities that both depend on the weights are multiplied, such as a gate multiplying a signal inside an LSTM.
- The sigmoid $\dfrac{1}{1+e^{-z}}$ and the softmax $\dfrac{e^{z_i}}{\sum_j e^{z_j}}$ are quotients. Part 4 differentiates the sigmoid.

> **HIDDEN MEANING**
> Two of these rules describe how learning itself works. Because rates add, the gradient of the total loss is the sum of every example's gradient: each example casts a vote on how every weight should move, and a random mini-batch is a fair sample of that vote. That is what makes training on enormous datasets possible. And because of the product rule, a factor near zero blocks the gradient to whatever it multiplies. That is *gating*: when an LSTM gate is closed, the signal behind it gets almost no learning signal.

Try it

1. x^3
2. $7x^4$
3. $x^5 + 3x^2$
4. $8x^7 - 4x^3 + 9$
5. $\dfrac{1}{x^2}$
6. $x^2(x+3)$, using the product rule
7. $(x^3+2)\,x^2$, using the product rule
8. $\dfrac{x^3}{x+2}$
9. $\dfrac{x^2+1}{x}$

▼ ANSWERS

1. $3x^2$
2. $28x^3$
3. $5x^4 + 6x$
4. $56x^6 - 12x^2$
5. $-2x^{-3} = -\dfrac{2}{x^3}$
6. $2x(x+3) + x^2 = 3x^2 + 6x$
7. $3x^2 \cdot x^2 + (x^3+2) \cdot 2x = 5x^4 + 4x$
8. $\dfrac{3x^2(x+2) - x^3}{(x+2)^2} = \dfrac{2x^3 + 6x^2}{(x+2)^2}$
9. $\dfrac{2x \cdot x - (x^2+1)}{x^2} = \dfrac{x^2-1}{x^2}$. Or split it first: $x + x^{-1}$ has derivative $1 - x^{-2}$, which is the same thing.

Part 3: The chain rule

When one function sits inside another, their rates of change multiply. This one rule, applied over and over, is backpropagation.

Functions inside functions

Look at

$$y = (3x + 2)^5$$

Two things happen in order. First x is turned into $u = 3x + 2$. Then u is raised to the fifth power, $y = u^5$. Draw it as a chain:

$$x \longrightarrow u \longrightarrow y$$

x never acts on y directly. It moves u, and u moves y.

Multiply the rates

Each link in the chain has its own rate of change:

- $\dfrac{du}{dx} = 3$. Every unit of change in x moves u by 3.
- $\dfrac{dy}{du} = 5u^4$. At $u = 2$ this is $5 \cdot 2^4 = 80$, so a small change in u moves y about 80 times as much.

Pass a small nudge along the chain. It is multiplied by 3 at the first link and by 80 at the second, so it arrives multiplied by 240. Gears work the same way: if the first gear turns the second three times per turn, and the second turns the third eighty times per turn, then the first turns the third 240 times.

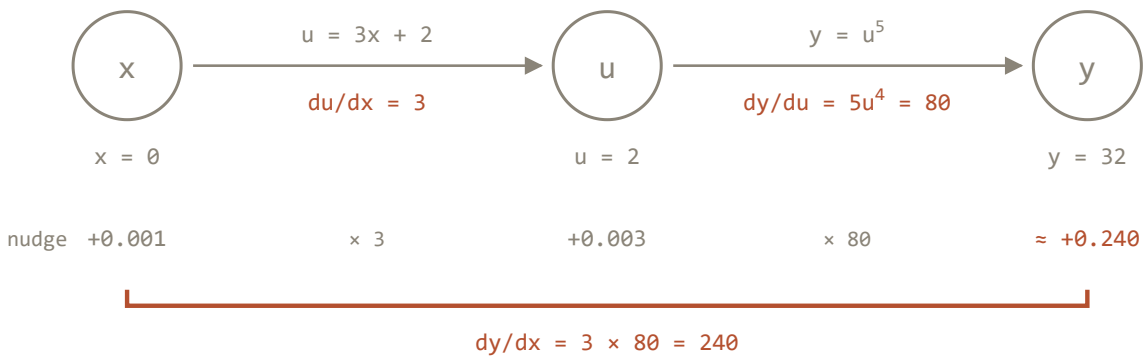


Figure 1. A nudge of 0.001 in x becomes 0.003 in u, then about 0.240 in y. The rates multiply.

That is the chain rule:

$$\boxed{\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx}}$$

> KEY IDEA
> When functions are chained, their rates of change multiply.

For this example:

$$\frac{dy}{dx} = 5u^4 \cdot 3 = 15\,(3x+2)^4$$

Check it with numbers. At $x = 0$, $u = 2$ and $y = 32$, and the formula predicts a slope of $15 \cdot 2^4 = 240$. Nudge x to 0.001: then $u = 2.003$ and $y = 2.003^5 \approx 32.2407$. The output rose by about 0.2407, which is 240×0.001.

A note on the notation. In $\frac{du}{dx}$, the bottom is what you nudge and the top is what you watch. So $\frac{du}{dx}$ means "nudge x, watch u", and $\frac{dy}{du}$ means "nudge u, watch y". Written this way the chain rule looks like fractions cancelling. They don't literally cancel, but the notation was designed so that you can treat them as if they do, which makes the rule hard to get wrong.

Longer chains

Nothing limits a chain to two links. For $y = (x^2+3)^4$, name every step:

$$x \to v = x^2 \to u = v + 3 \to y = u^4$$

and multiply all the rates:

$$\begin{aligned}\frac{dy}{dx} &= \frac{dy}{du} \cdot \frac{du}{dv} \cdot \frac{dv}{dx} \\ &= 4u^3 \cdot 1 \cdot 2x = 8x\,(x^2+3)^3\end{aligned}$$

Naming the intermediate steps is not a crutch for beginners. It is exactly how software that computes derivatives works, as Part 7 shows.

A recipe

1. Find the outermost operation.
2. Differentiate it, leaving whatever is inside untouched.
3. Multiply by the derivative of the inside.

4. If the inside has an inside of its own, repeat.

The most common mistake is in step 2. Differentiating $(2x+5)^4$, the power drops by one; it does not disappear:

$$\begin{aligned}\frac{d}{dx}(2x+5)^4 &= 4\,(2x+5)^3 \cdot 2 \\ &= 8\,(2x+5)^3\end{aligned}$$

and not $8\,(2x+5)$.

Combined with the other rules

Product and chain: $y = x^2(3x+1)^5$ is a product, and its second factor needs the chain rule.

$$y' = 2x(3x+1)^5 + 15x^2(3x+1)^4$$

Quotient and chain: for $y = \dfrac{(2x+1)^3}{x^2+1}$, the top needs the chain rule, $\dfrac{d}{dx}(2x+1)^3 = 6(2x+1)^2$, and the bottom's derivative is $2x$. So the top of the quotient rule is $6(2x+1)^2(x^2+1) - 2x(2x+1)^3$. Take out the common factor $(2x+1)^2$ and it simplifies to

$$y' = \frac{2(2x+1)^2(x^2-x+3)}{(x^2+1)^2}$$

Why this is the most important rule in AI

A neural network is a long chain of functions. The input x passes through layer after layer, each producing an output a_1, a_2, a_3, and the last output feeds the loss L:

$$x \to a_1 \to a_2 \to a_3 \to L$$

To train a weight w in the first layer you need the derivative of the loss with respect to w, and that weight affects the loss only through every layer after it. The chain rule says what to do: multiply the local rates along the path.

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial a_3} \cdot \frac{\partial a_3}{\partial a_2} \cdot \frac{\partial a_2}{\partial a_1} \cdot \frac{\partial a_1}{\partial w}$$

The curly ∂ is used when a function has more than one input, which Part 7 explains. For now, read it as d.

Backpropagation is this product computed backwards from the loss, so that the shared part of the product is worked out once and reused for every weight before it. It is the chain rule, organised.

The same product explains a famous problem. If each local rate in a 20-layer chain is about 0.5, the product is $0.5^{20} \approx 0.000001$: the gradient *vanishes*, and the early layers barely learn. If each rate is about 1.5, the

product is $1.5^{20} \approx 3300$, and the gradient *explodes*.[1] Much of modern network design, including residual connections, the gates of an LSTM and careful weight initialisation, exists to keep that product close to 1.

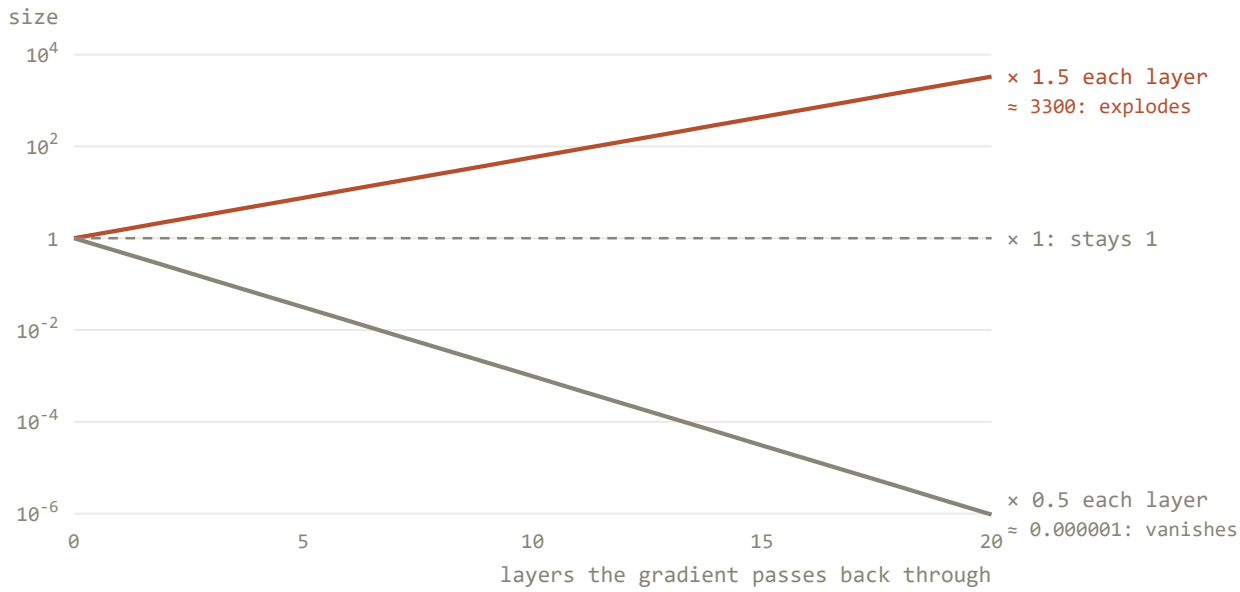


Figure 2. The product of the local rates as a gradient passes back through 20 layers, on a log scale. At 0.5 per layer it falls to about one millionth; at 1.5 it grows to about 3300.

> **HIDDEN MEANING**
>
> The chain rule is *credit assignment*. A weight deep inside a network never touches the loss; its share of the blame is the product of every local rate between it and the output. Influence multiplies rather than adds, so one weak link anywhere silences everything before it. Read this way, vanishing gradients are a statement about memory: in a recurrent network, an input from 100 steps ago can only shape learning if its influence survives 100 multiplications.

Try it

1. $(2x+1)^3$
2. $(x^2+3)^4$
3. $(x^2+1)^3$
4. $x^3(2x+5)^4$
5. $\dfrac{(2x+1)^3}{x^2+1}$
6. $\sqrt{1+x^2}$

▼ ANSWERS

1. $3(2x+1)^2 \cdot 2 = 6(2x+1)^2$
2. $4(x^2+3)^3 \cdot 2x = 8x(x^2+3)^3$
3. $3(x^2+1)^2 \cdot 2x = 6x(x^2+1)^2$
4. $3x^2(2x+5)^4 + x^3 \cdot 8(2x+5)^3$
5. The top is $6(2x+1)^2(x^2+1) - 2x(2x+1)^3$ and the bottom is $(x^2+1)^2$. Simplified: $\dfrac{2(2x+1)^2(x^2-x+3)}{(x^2+1)^2}$.
6. Write it as $(1+x^2)^{1/2}$. Then $\frac{1}{2}(1+x^2)^{-1/2} \cdot 2x = \dfrac{x}{\sqrt{1+x^2}}$.

---

1. Bengio, Simard & Frasconi (1994), “Learning long-term dependencies with gradient descent is difficult”, *IEEE Transactions on Neural Networks*, 5(2). ↩

Part 4: Exponentials, logarithms and sines

The derivatives of the exponential, the logarithm, sine and cosine, and why they matter. They are the raw material of sigmoid, softmax, cross-entropy and positional encodings.

The function that is its own slope

$$\frac{d}{dx}\, e^{x} = e^{x}$$

At every point, the slope of e^{x} equals its height. The bigger it gets, the faster it grows. That is what exponential growth means: growth in proportion to size, like money left to compound. The number $e \approx 2.718$ is the one base for which the slope and the height are exactly equal.

With the chain rule, whatever sits in the exponent contributes its own derivative:

$$\frac{d}{dx}\, e^{u} = e^{u}\, \frac{du}{dx}$$

$$\frac{d}{dx}\, e^{3x+2} = 3\, e^{3x+2}$$

$$\frac{d}{dx}\, e^{2x} = 2\, e^{2x}$$

$$\frac{d}{dx}\, e^{x^2+3} = 2x\, e^{x^2+3}$$

The base stays e. The factor in front comes from differentiating the exponent.

The logarithm asks “what power?”

Since $2^3 = 8$, the logarithm of 8 to base 2 is 3: $\log_2 8 = 3$. The natural logarithm, $\ln$, uses base e, and it undoes the exponential:

$$\ln(e^{x}) = x \qquad e^{\ln x} = x$$

Its derivative is

$$\frac{d}{dx}\, \ln x = \frac{1}{x}$$

The reason is a picture. The graph of $\ln x$ is the graph of e^{x} reflected in the line $y = x$. Reflecting swaps rise and run, so every slope turns into its reciprocal. The point $(x, \ln x)$ on the log curve matches the point $(\ln x, x)$ on the exponential curve, where the slope is $e^{\ln x} = x$. So the log curve’s slope is $1/x$.

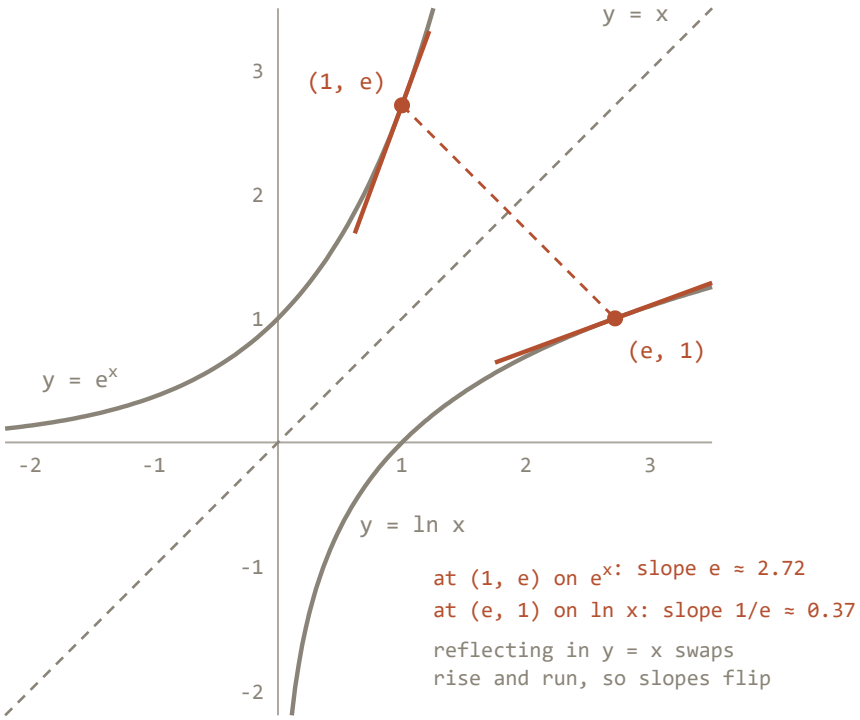


Figure 1. e^x and $\ln x$ are mirror images in the line $y = x$. At $(1, e)$ the exponential has slope e; at the matching point $(e, 1)$ the logarithm has slope $1/e$.

With the chain rule:

$$\frac{d}{dx}\ln u = \frac{1}{u}\frac{du}{dx}$$

$$\frac{d}{dx}\ln(x^2+1) = \frac{2x}{x^2+1}$$
$$\frac{d}{dx}\ln(3x+2) = \frac{3}{3x+2}$$

Two details are worth knowing:

- $\ln x$ only exists for $x > 0$. The version that works on both sides of zero is $\ln|x|$, and its derivative is also $1/x$. For example, $\ln(x^2) = 2\ln|x|$, whose derivative is $2/x$.
- Logarithms turn products into sums, $\ln(ab) = \ln a + \ln b$, and powers into products, $\ln(a^b) = b\ln a$. Both turn hard derivatives into easy ones.

Other bases

$$\frac{d}{dx}a^x = a^x \ln a$$

The $\ln a$ comes from rewriting the base as a power of e. Since $a = e^{\ln a}$, we have $a^x = e^{x \ln a}$, and the chain rule gives $e^{x \ln a} \cdot \ln a = a^x \ln a$. The factor $\ln a$ compares base a with base e: $\ln 2 \approx 0.69$, so 2^x grows more slowly than its own size, while $\ln 5 \approx 1.61$. Because $\ln e = 1$, the rule for e^x is the special case.

Logarithmic differentiation

$y = x^x$ fits none of the rules so far. The power rule needs a fixed exponent, and the exponential rule needs a fixed base; here both vary. Take $\ln$ of both sides to bring the exponent down:

$$\ln y = x \ln x$$

Now differentiate both sides with respect to x. On the right, the product rule gives $\ln x + x \cdot \frac{1}{x} = \ln x + 1$. On the left, y depends on x, so the chain rule gives $\frac{1}{y} \frac{dy}{dx}$. Part 5 explains this step in detail.

$$\frac{1}{y} \frac{dy}{dx} = \ln x + 1$$

Multiply both sides by $y = x^x$:

$$\frac{dy}{dx} = x^x \left(\ln x + 1 \right)$$

Sine, cosine and tangent

$$\begin{aligned} \frac{d}{dx} \sin x &= \cos x \\ \frac{d}{dx} \cos x &= -\sin x \\ \frac{d}{dx} \tan x &= \sec^2 x \end{aligned}$$

Picture a point going round a circle at steady speed, with $\sin t$ as its height. At the top of the circle the point is moving sideways, so its height has momentarily stopped changing, and that is where $\cos t = 0$. As it crosses the middle it is moving straight up or down, so its height changes fastest, and $\cos t = \pm 1$. The rate of change of the height follows the cosine.

The tangent follows from the quotient rule, since $\tan x = \sin x / \cos x$. The top of the quotient rule is $\cos x \cdot \cos x - \sin x \cdot (-\sin x) = \cos^2 x + \sin^2 x$, which is 1 by Pythagoras:

$$\begin{aligned} \frac{d}{dx} \tan x &= \frac{\cos^2 x + \sin^2 x}{\cos^2 x} \\ &= \frac{1}{\cos^2 x} = \sec^2 x \end{aligned}$$

The chain rule works as always: $\frac{d}{dx} \sin 3x = 3 \cos 3x$.

Where these show up in AI

Sigmoid. The function

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

squashes any number into the range 0 to 1. It turns a score into a probability, and it builds the gates of an LSTM. Its derivative needs only the rules so far. Write $\sigma = (1 + e^{-z})^{-1}$ and use the power rule with the chain rule:

$$\begin{aligned} \sigma'(z) &= -(1 + e^{-z})^{-2} \cdot \left(-e^{-z}\right) \\ &= \frac{e^{-z}}{(1 + e^{-z})^2} \\ &= \sigma(z)\left(1 - \sigma(z)\right) \end{aligned}$$

The tidy final form means a network that has already computed σ gets its derivative almost for free. Its largest value is 0.25, at $z = 0$. Stack many sigmoids and the chain rule multiplies many numbers no bigger than 0.25, which is one reason deep stacks of sigmoids were hard to train (Part 3). Most modern networks use the ReLU, $\max(0, z)$, whose slope is exactly 1 for every positive input.

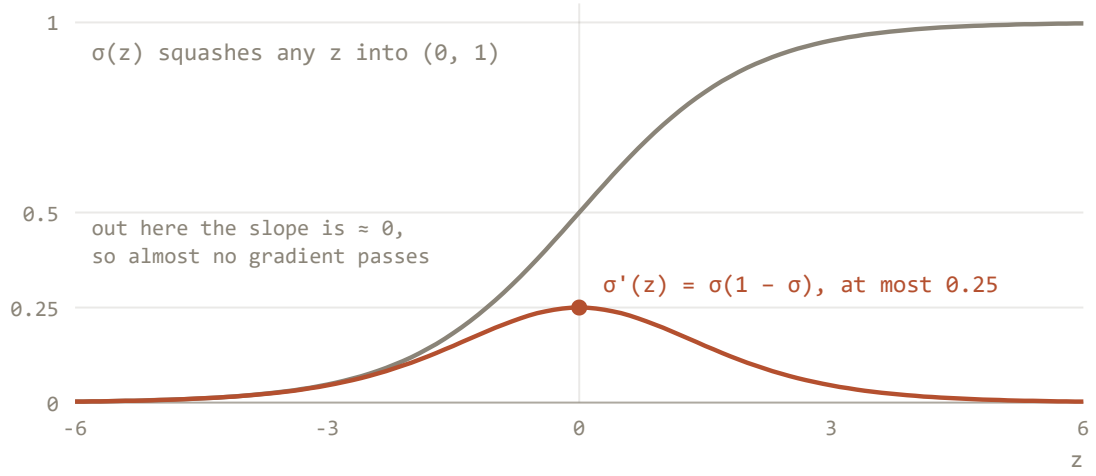


Figure 2. The sigmoid and its slope. The slope is never more than 0.25, and far from zero it is almost 0, so very little gradient gets through.

Softmax. A language model ends by scoring every possible next token, then turning the scores z_i into probabilities:

$$p_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

The exponential makes every score positive, and dividing by the sum makes them add up to 1.

Cross-entropy. The loss most models train on is the negative log of the probability the model gave the right answer, $L = -\ln p$. If that probability is 0.9 the loss is 0.105; if it is 0.01 the loss is 4.6. The derivative, $-1/p$, is huge when p is tiny, so confident mistakes are corrected hardest. Combined with softmax, the derivative of this loss with respect to each score z_i comes out remarkably simple: $p_i - 1$ for the right answer and p_i for every other. Models also work with log-probabilities rather than probabilities because multiplying thousands of small probabilities underflows to zero, while adding their logarithms does not.

Sines and cosines. The original Transformer told the network where each word sits in a sentence by adding sine and cosine waves of different frequencies to it.[1] Many current language models use rotary position embeddings instead, which rotate pairs of components of each query and key vector by an angle that grows with position, and rotations are built from $\sin$ and $\cos$.[2] Networks that use $\sin$ as their activation function can represent images, sound and 3D shapes as smooth functions.[3] In all of these, training needs the derivatives above.

> HIDDEN MEANING
>
> The exponential and the logarithm translate between two ways of thinking. The exponential turns adding into multiplying: raise one softmax score by 1 and its odds against every other option are multiplied by e, so a network's scores are really log-odds. The logarithm turns multiplying into adding, so evidence that multiplies as probability adds as log-probability, and $-\ln p$ measures *surprise*. Together they make the learning signal $p_i - y_i$, prediction minus truth: confident and right gives almost no gradient, confident and wrong gives a large one. Networks learn from being surprised.

Try it

1. e^{2x}
2. e^{x^2+3}
3. 4^x
4. $\ln(3x+2)$
5. x^x
6. $\cos 2x$
7. $\sin(x^2)$
8. $\tan(5x+1)$

▼ ANSWERS

1. $2e^{2x}$. The base stays e; the 2 comes from the exponent.
2. $2x\, e^{x^2+3}$
3. $4^x \ln 4$
4. $\dfrac{3}{3x+2}$
5. $x^x(\ln x + 1)$
6. $-2\sin 2x$
7. $2x\cos(x^2)$
8. $5\sec^2(5x+1)$

---

1. Vaswani et al. (2017), “Attention is all you need”, section 3.5. ↩
2. Su et al. (2021), “RoFormer: Enhanced Transformer with rotary position embedding”. ↩
3. Sitzmann et al. (2020), “Implicit neural representations with periodic activation functions” (SIREN). ↩

Part 5: Implicit differentiation

Sometimes y can't be written on its own. You can still find its slope by remembering that y depends on x, which turns every y-term into a small chain rule.

When y won't stand alone

So far every function came as $y = f(x)$. Sometimes x and y are mixed together:

$$x^2 + y^2 = 25$$

That is a circle of radius 5. You could solve for $y = \pm\sqrt{25 - x^2}$ and differentiate, but it is clumsy, and for many equations solving for y is impossible. Implicit differentiation skips that step: differentiate both sides of the equation as they stand.

The one idea: y depends on x

Even when it isn't written as $y = f(x)$, y is still a function of x. Move along the circle and y changes as x does. So any term containing y hides a chain:

$$x \longrightarrow y \longrightarrow y^2$$

Differentiating y^2 with respect to x takes two steps:

1. x changes, and y changes with it, at rate $\dfrac{dy}{dx}$.
2. y changes, and y^2 changes with it, at rate $2y$.

Multiply the rates, as in Part 3:

$$\frac{d}{dx}\, y^2 = 2y\, \frac{dy}{dx}$$

With numbers: suppose that at some point $\dfrac{dy}{dx} = 3$ and $y = 2$. A small change in x makes y change 3 times as much, and that makes y^2 change $2y = 4$ times as much again. Overall, y^2 changes 12 times as fast as x.

Compare the two questions. The first nudges y itself; the second nudges x, which then moves y:

$$\frac{d}{dy}\, y^2 = 2y$$
$$\frac{d}{dx}\, y^2 = 2y\, \frac{dy}{dx}$$

The extra $\dfrac{dy}{dx}$ is the cost of going through y. Every y-term carries one.

The circle

Differentiate both sides of $x^2 + y^2 = 25$ with respect to x. The right side is a constant, so its derivative is 0:

$$2x + 2y\,\frac{dy}{dx} = 0 \quad \Rightarrow \quad \frac{dy}{dx} = -\frac{x}{y}$$

Check it at the point $(3, 4)$, where the slope is $-\frac{3}{4}$. The radius from the centre to $(3, 4)$ has slope $\frac{4}{3}$, and a tangent to a circle is perpendicular to its radius. Perpendicular slopes multiply to -1, and indeed $-\frac{3}{4} \times \frac{4}{3} = -1$. At the top of the circle, $(0, 5)$, the slope is 0: flat, as it should be.

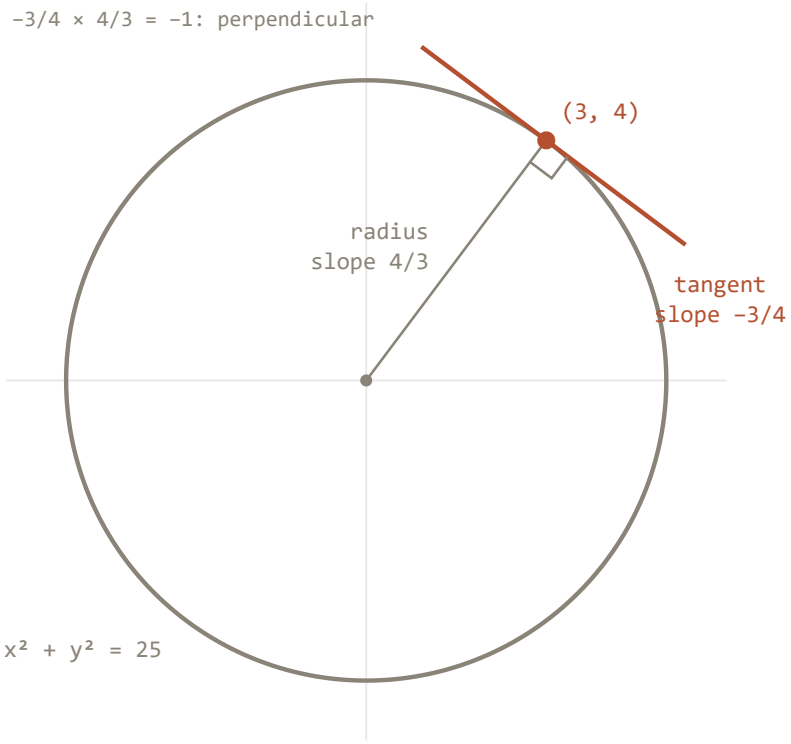


Figure 1. The tangent at (3, 4) has slope −3/4, at right angles to the radius, whose slope is 4/3. Implicit differentiation found it without ever solving for y.

With a product

For $x^2 + xy + y^2 = 7$, the middle term is a product of two things that both change with x, so it needs the product rule:

$$\frac{d}{dx}(xy) = 1 \cdot y + x\,\frac{dy}{dx}$$

Differentiate every term:

$$2x + y + x\,\frac{dy}{dx} + 2y\,\frac{dy}{dx} = 0$$

Collect the $\dfrac{dy}{dx}$ terms:

$$\frac{dy}{dx}\,(x + 2y) = -(2x + y)$$

and solve:

$$\frac{dy}{dx} = -\frac{2x + y}{x + 2y}$$

This is also the move that appeared on the left-hand side in Part 4, where $\ln y$ became $\frac{1}{y}\,\frac{dy}{dx}$.

Where this shows up in AI

Some models define their output by an equation it has to satisfy rather than by a formula. A *deep equilibrium model* applies the same layer over and over until the output stops changing, reaching a fixed point $z = f(z, x)$.[1] Training it needs the derivative of z with respect to the input and the weights, yet z is never written as a formula. Implicit differentiation of $z = f(z, x)$ gives that derivative directly, without storing the many steps it took to reach the fixed point. The same technique gives gradients through an optimisation problem solved inside a model.

Fixed points matter for memory too. An associative memory such as a Hopfield network recalls a stored pattern by settling into a fixed point from a partial cue.

> HIDDEN MEANING
> Some answers are defined by a condition rather than a recipe: *the state where the system stops changing*. Implicit differentiation says you can work out how such an answer moves when its inputs move without knowing, or storing, how the system got there. For memory this matters: a network that recalls by settling into a fixed point can be trained through that fixed point directly, an idea that goes back to recurrent backpropagation in the 1980s.[2]

Try it

1. Find $\dfrac{d}{dx}\,y^3$.
2. For $x^2 + y^2 = 25$, find the slope at $(-4, 3)$.
3. For $x^2 + xy + y^2 = 7$, find the slope at the point $(1, 2)$.

▼ ANSWERS

1. $3y^2 \dfrac{dy}{dx}$
2. $-\dfrac{x}{y} = -\dfrac{-4}{3} = \dfrac{4}{3}$
3. $-\dfrac{2 \cdot 1 + 2}{1 + 2 \cdot 2} = -\dfrac{4}{5}$. The point is on the curve, since $1 + 2 + 4 = 7$.

---

1. Bai, Kolter & Koltun (2019), "Deep equilibrium models", *NeurIPS*. ↩
2. Pineda (1987), "Generalization of back-propagation to recurrent neural networks", *Physical Review Letters*, 59(19); Almeida (1987), "A learning rule for asynchronous perceptrons with feedback in a combinatorial environment", *IEEE First International Conference on Neural Networks*. ↩

Part 6: The second derivative

The derivative of the derivative says how the slope itself is changing. It describes the shape of a curve, and in training it decides how large a step is safe.

The rate of change of the rate of change

The first derivative says how fast a function is changing. Differentiate again and you learn how fast *that* is changing:

$$y = x^3 \;\rightarrow\; y' = 3x^2 \;\rightarrow\; y'' = 6x$$

The second derivative is written y'' or $\dfrac{d^2y}{dx^2}$. If y is the position of a car, y' is its speed and y'' is its acceleration: how hard it is speeding up or braking.

It is usually described in terms of slope because y' *is* the slope of the curve at each point, so y'' is the rate at which the slope changes as you move along. For $y = x^3$ the slope is 3 at $x = 1$ and 12 at $x = 2$, and $y'' = 6x$ measures how quickly it climbs between them.

What it says about shape

- $y'' > 0$: the slope is increasing, and the curve bends upward like a cup.
- $y'' < 0$: the slope is decreasing, and the curve bends downward like a cap.

This settles what kind of flat point you are at. Where $y' = 0$:

- if $y'' > 0$, you are at the bottom of a cup, a **minimum**;
- if $y'' < 0$, you are at the top of a cap, a **maximum**;
- if $y'' = 0$, the second derivative can't tell. For $y = x^3$ at $x = 0$, both derivatives are zero and the point is neither: the curve flattens for a moment, then keeps rising.

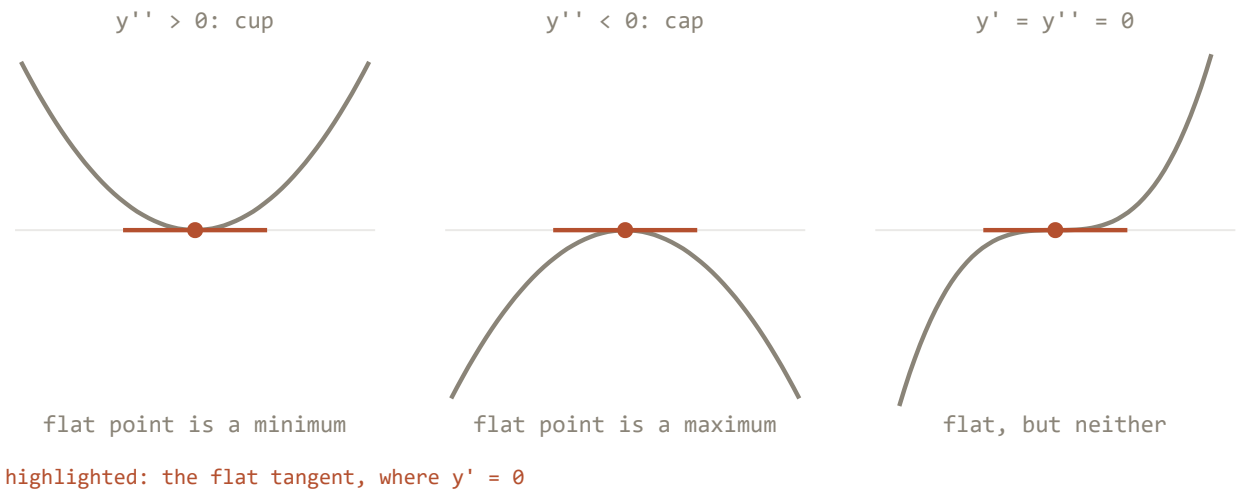


Figure 1. Three flat points. The second derivative tells them apart: a cup is a minimum, a cap is a maximum, and x^3 is neither.

Second derivatives use every rule

For $y = e^{x^2}$, the chain rule gives $y' = 2x\,e^{x^2}$. That is a product, so the second derivative needs the product rule and the chain rule again:

$$\begin{aligned} y'' &= 2\,e^{x^2} + 2x \cdot 2x\,e^{x^2} \\ &= (2 + 4x^2)\,e^{x^2} \end{aligned}$$

Where this shows up in AI: how big a step is safe

Training moves a weight against the slope of the loss:

$$w \;\leftarrow\; w - \eta\,L'(w)$$

Here η (eta) is the learning rate, the size of the step. How large can it be? That depends on the curvature, L''.

Take the simplest bowl, $L(w) = \frac{a}{2}\,w^2$. Its slope is $L' = aw$ and its curvature is $L'' = a$. One step gives

$$w \;\leftarrow\; w - \eta a w = (1 - \eta a)\,w$$

so every step multiplies w by $1 - \eta a$. Three things can happen:

- $\eta a < 1$: w shrinks steadily toward the minimum at 0.
- $1 < \eta a < 2$: each step overshoots to the other side, but lands closer, so it still settles.
- $\eta a > 2$: each step lands farther out than the last, and training *diverges*.

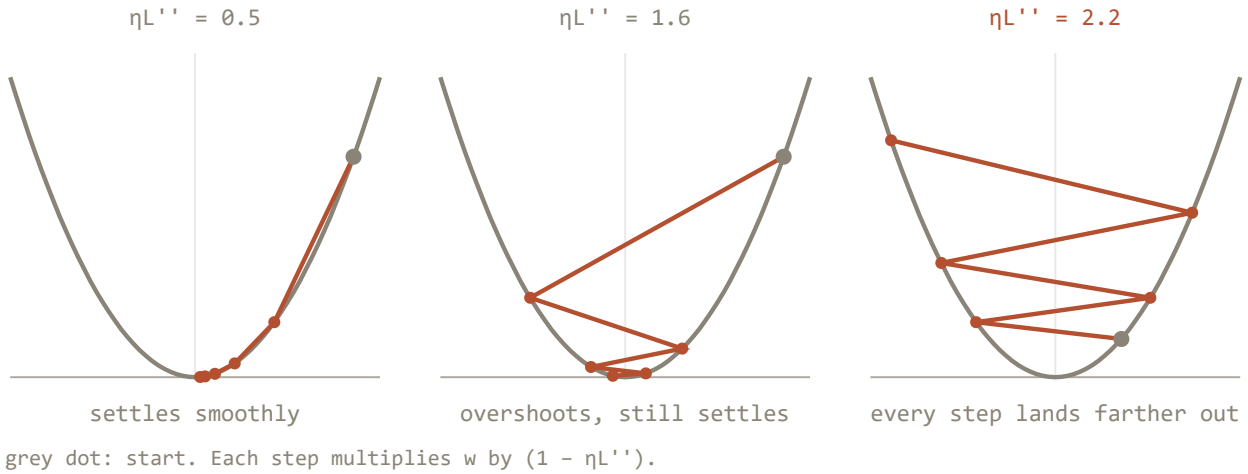


Figure 2. Gradient descent on L = w² with three step sizes. Each step multiplies w by (1 − ηL''). Below 1 it settles smoothly, between 1 and 2 it zig-zags in, above 2 it flies off.

So a sharply curved loss demands a small learning rate: η has to stay below $2/L''$. In the one-weight example on the topic page, $L'' = 8$, so any learning rate above 0.25 would have made w swing wider at every step. In a real network the curvature is different in every direction (Part 9), and the most sharply curved direction sets the limit. This is why a learning rate that is too high makes the loss blow up instead of fall.

> **HIDDEN MEANING**
> The first derivative says which way to go; the second says how far you can trust it. Where the curvature is high the slope changes quickly, so the straight-line picture behind every gradient step breaks down within a short distance. Curvature also says something about robustness: a minimum at the bottom of a sharp, narrow valley is sensitive to small changes in the weights, while one in a wide, flat valley is not. Whether flatter minima generalise better is an influential and still debated idea.[1]

Try it

1. For $y = x^4$, find y' and y''.
2. For $y = x^3 - 3x$, find the flat points and say whether each is a minimum or a maximum.
3. For $L(w) = 5w^2$, what is the largest learning rate that does not diverge?

> **▼ ANSWERS**
>
> 1. $y' = 4x^3$ and $y'' = 12x^2$.
> 2. $y' = 3x^2 - 3 = 0$ at $x = \pm 1$, and $y'' = 6x$. At $x = 1$, $y'' = 6 > 0$: a minimum. At $x = -1$, $y'' = -6 < 0$: a maximum.
> 3. $L'' = 10$, so η must stay below $2/10 = 0.2$.

1. Hochreiter & Schmidhuber (1997), “Flat minima”, *Neural Computation*, 9(1); Keskar et al. (2017), “On large-batch training for deep learning: generalization gap and sharp minima”, *ICLR*; Dinh et al. (2017), “Sharp minima can generalize for deep nets”, *ICML*. ↩

Part 7: Partial derivatives and one neuron

When a function has many inputs, change one at a time and hold the rest still. That is enough to work out, by hand, how a neuron's error depends on its weight and bias.

Functions of many inputs

In AI a function almost never has a single input. A network's loss depends on every one of its weights at once. Start with two inputs:

$$z = x^2 + 3xy + y^2$$

Think of z as your height on a hillside, x as how far east you are and y as how far north. Face east and take a small step: how fast do you climb? Face north instead: how fast now? These are two different questions with two different answers, and each answer is a **partial derivative**.

The rule is short: **differentiate with respect to one variable, and treat every other variable as a constant.**

With respect to x, holding y fixed:

$$\frac{\partial z}{\partial x} = 2x + 3y + 0$$

With y frozen, the middle term $3xy$ is just the constant $3y$ times x, so its derivative is $3y$. The last term, y^2, doesn't involve x at all, so it contributes 0.

With respect to y, holding x fixed:

$$\frac{\partial z}{\partial y} = 0 + 3x + 2y$$

The curly ∂ is a reminder that other variables exist and are being held still.

The chain rule works exactly as before. For $z = (2x + 3y)^3$, the outer cube gives $3(2x + 3y)^2$, and the inside contributes 2 when you nudge x and 3 when you nudge y:

$$\frac{\partial z}{\partial x} = 6(2x + 3y)^2$$
$$\frac{\partial z}{\partial y} = 9(2x + 3y)^2$$

One neuron, one example

The simplest model is a single neuron: it takes an input x, multiplies it by a weight w and adds a bias b. Its error on one example whose correct answer is y is

$$L = (wx + b - y)^2$$

During training, x and y come from the data, so they are fixed. The weight and the bias are what get adjusted. So we want the partial derivatives with respect to w and b, and the chain rule gives them:

$$\frac{\partial L}{\partial w} = 2\,(wx + b - y) \cdot x$$
$$\frac{\partial L}{\partial b} = 2\,(wx + b - y) \cdot 1$$

Read them. $wx + b - y$ is the error, prediction minus target. The weight's gradient is the error times the input: a weight attached to a large input gets a large share of the blame, and a weight whose input was zero gets none, because it had no effect on this prediction. The bias always gets the error itself.

You can also ask how the loss depends on the input:

$$\frac{\partial L}{\partial x} = 2\,(wx + b - y) \cdot w$$

In a deep network, this is how the gradient travels back to the previous layer, whose output is this layer's input. It is also how *adversarial examples* are found: the gradient with respect to an image's pixels shows which tiny changes to the image would fool a classifier the most.[1]

The computational graph

Rather than differentiate one big formula, break the computation into named steps:

$$u = wx + b \qquad L = (u - y)^2$$

Each step has a small local derivative:

$$\frac{\partial L}{\partial u} = 2(u - y)$$
$$\frac{\partial u}{\partial w} = x$$
$$\frac{\partial u}{\partial b} = 1$$

and the chain rule joins them:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial u} \cdot \frac{\partial u}{\partial w} = 2(u - y)\,x$$

That is the same answer as before. No step needs to know about any other; each knows only its own small derivative, and the chain rule stitches them together. This picture, with values flowing forward through named steps and derivatives flowing back, is called a **computational graph**.

A deeper chain, by hand

Add one more step:

$$
\begin{aligned}
u &= wx + b \\
v &= u^2 \\
L &= (v - y)^2
\end{aligned}
$$

The bias reaches the loss through a chain, $b \to u \to v \to L$, so

$$
\begin{aligned}
\frac{\partial L}{\partial b} &= \frac{\partial L}{\partial v}\,\frac{\partial v}{\partial u}\,\frac{\partial u}{\partial b} \\
&= 2(v - y) \cdot 2u \cdot 1 \\
&= 4u\,(v - y)
\end{aligned}
$$

Now put in numbers: the data are $x = 2$ and $y = 3$, and the starting guess is $w = 0.5, b = 0$.

Forward pass, computing values from the input to the loss:

STEP	VALUE
$u = wx + b$	1
$v = u^2$	1
$L = (v - y)^2$	4

Backward pass, computing derivatives from the loss back to the parameters:

DERIVATIVE	COMPUTED AS	VALUE
$\partial L / \partial v$	$2(v - y)$	-4
$\partial L / \partial u$	$(-4) \cdot 2u$	-8
$\partial L / \partial b$	$(-8) \cdot 1$	-8
$\partial L / \partial w$	$(-8) \cdot x$	-16

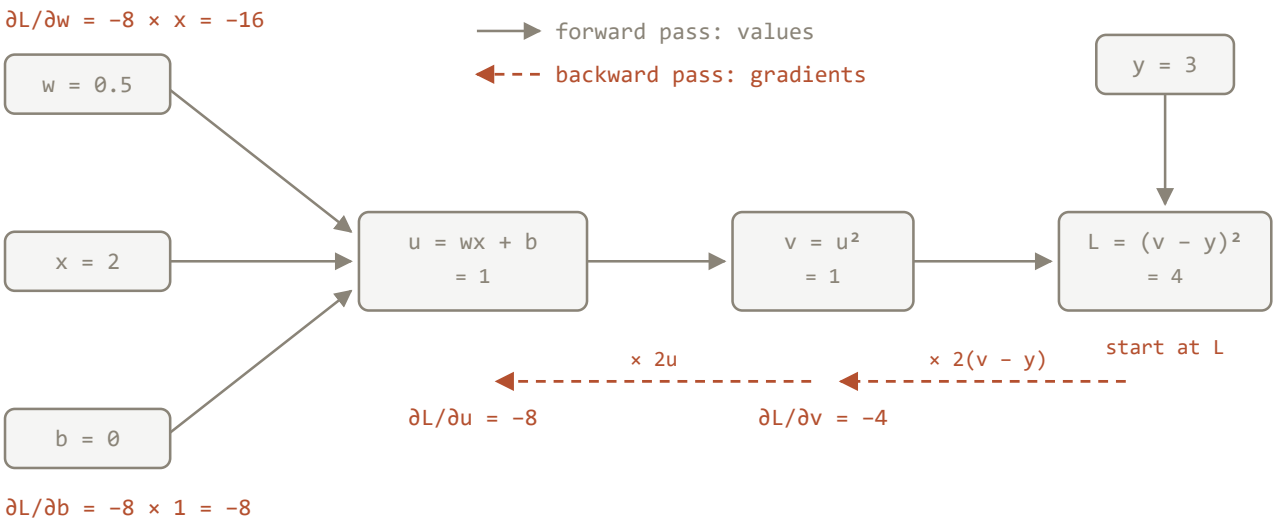


Figure 1. The same computation as a graph. Values flow forward to the loss; gradients start at the loss and flow back, each step multiplying by one local derivative.

Notice that $\partial L/\partial u = -8$ was computed once and used twice, for both b and w. In a network with millions of weights, that reuse is the whole trick. One backward pass produces every gradient, at a cost comparable to one forward pass. That is **backpropagation**.

Both gradients are negative, so increasing w and b will lower the loss. Part 8 takes that step.

What `loss.backward()` does

Deep-learning frameworks do exactly this. Here is the same neuron in PyTorch:

```
import torch

x, y = 2.0, 3.0
w = torch.tensor(0.5, requires_grad=True)
b = torch.tensor(0.0, requires_grad=True)

# Forward pass: L = 4
u = w * x + b
v = u ** 2
L = (v - y) ** 2

# Backward pass: the chain rule,
# from L back to w and b
L.backward()
print(w.grad, b.grad)
# tensor(-16.) tensor(-8.)
```

The framework records each step of the forward computation as a graph, then walks it backwards multiplying local derivatives, just as in the table above. It uses no calculus beyond what you have just done by hand.

> **HIDDEN MEANING**
> "Hold everything else fixed" is an assumption, not a fact. It judges each weight as if it acted alone, which is only true for tiny moves. In a real step every weight moves at once and their effects interact, which is one more reason steps must stay small. The neuron's gradient also shows the shape of learning: the update to a weight is the signal coming in times the error coming back. That has the same form as Hebb's rule, popularly "cells that fire together wire together", with the error taking the place of the second cell's activity.

Try it

1. For $z = 4x^2 + 5xy + 3y^2$, find $\partial z/\partial x$ and $\partial z/\partial y$.
2. For $z = x^2y + 3xy^2 + 2y$, find both partial derivatives.
3. For $z = (x + y)^2$, find both partial derivatives without expanding.
4. With $u = wx + b$ and $v = u^2$, find $\partial v/\partial w$.
5. In the deeper chain, find $\partial L/\partial w$ as a formula, then check it against the table.

> **▼ ANSWERS**
> 1. $8x + 5y$ and $5x + 6y$
> 2. $2xy + 3y^2$ and $x^2 + 6xy + 2$
> 3. Both are $2(x + y)$, because x and y each appear inside the bracket with coefficient 1.
> 4. $2u \cdot x = 2ux$
> 5. $2(v - y) \cdot 2u \cdot x = 4ux\,(v - y)$. With the numbers above, $4 \cdot 1 \cdot 2 \cdot (1 - 3) = -16$.

---

1. Goodfellow, Shlens & Szegedy (2015), "Explaining and harnessing adversarial examples", *ICLR*. ↩

Part 8: The gradient, and which way is downhill

Collect a function's partial derivatives into one arrow and it points straight uphill. Step the other way and you have gradient descent, the update at the heart of training.

All the partial derivatives in one arrow

The **gradient** of a function collects its partial derivatives into a vector. It is written ∇f, read "grad f".

$$z = x^2 + y^2 \quad \Rightarrow \quad \nabla z = \begin{bmatrix} 2x \\ 2y \end{bmatrix}$$

For $z = 3x^2 + 2xy + y^2$:

$$\nabla z = \begin{bmatrix} 6x + 2y \\ 2x + 2y \end{bmatrix}$$

and at the point $(1, 2)$:

$$\nabla z(1, 2) = \begin{bmatrix} 10 \\ 6 \end{bmatrix}$$

There, z rises at rate 10 if you step in x, and at rate 6 if you step in y.

The gradient is an arrow you can draw at every point. Two properties make it useful: it points in the direction of steepest climb, and its length is how steep that climb is. The rest of this part shows why.

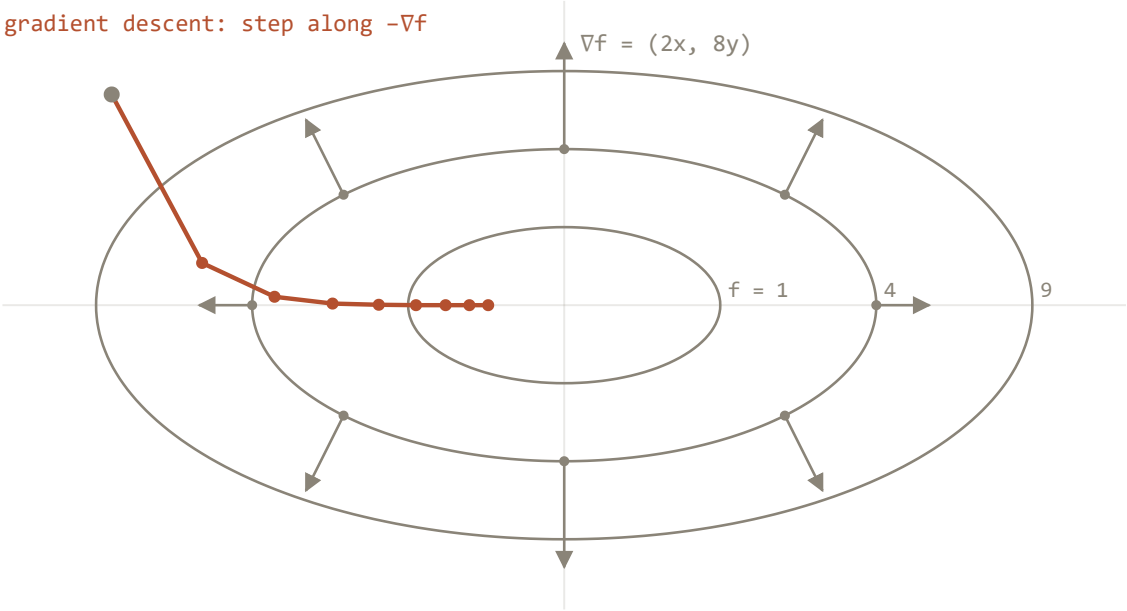


Figure 1. Contours of f = x² + 4y² with gradient arrows (2x, 8y). The arrows cross the contours at right angles and are longest where the contours are closest together. The highlighted path is gradient descent.

Climbing in any direction

A partial derivative measures the slope along one of the axes. The **directional derivative** measures it in any direction you choose. If $\mathbf{u}$ is a vector of length 1 pointing that way,

$$D_{\mathbf{u}} f = \nabla f \cdot \mathbf{u}$$

The dot is the dot product: multiply matching components and add them, $(a, b) \cdot (c, d) = ac + bd$.

With $\nabla f = (2, 8)$:

- East, $\mathbf{u} = (1, 0)$: $2 \cdot 1 + 8 \cdot 0 = 2$, which is just $\partial f / \partial x$.
- North, $\mathbf{u} = (0, 1)$: $2 \cdot 0 + 8 \cdot 1 = 8$, which is $\partial f / \partial y$.

Why the direction must have length 1

The directional derivative is meant to be *climb per unit of distance*. A longer direction vector would inflate the answer just by being longer, so it is first scaled to length 1, a **unit vector**.

The vector $(1, 1)$ means one step right and one step up. Those two steps form a right-angled triangle, and the diagonal is longer than either side: by Pythagoras its length is $\sqrt{1^2 + 1^2} = \sqrt{2} \approx 1.414$. Divide by that length to get a unit vector pointing the same way:

$$\hat{\mathbf{u}} = \left(\tfrac{1}{\sqrt{2}}, \tfrac{1}{\sqrt{2}} \right)$$

With $\nabla f = (2, 8)$ again, the directional derivative along it is

$$D_{\hat{\mathbf{u}}} f = \frac{2 + 8}{\sqrt{2}} = 5\sqrt{2} \approx 7.07$$

A worked example: $f(x, y) = x^2 + 4y^2$ at the point $(1, 1)$, in the direction of $(3, 4)$.

1. The gradient is $\nabla f = (2x, 8y)$, so $\nabla f(1, 1) = (2, 8)$.
2. The length of $(3, 4)$ is $\sqrt{9 + 16} = 5$, so the unit vector is $(0.6, 0.8)$.
3. The dot product is $2 \cdot 0.6 + 8 \cdot 0.8 = 1.2 + 6.4 = 7.6$.

Why the gradient points uphill

The dot product of two arrows also equals their lengths multiplied together, times the cosine of the angle θ between them. With a unit vector that gives

$$D_{\hat{\mathbf{u}}} f = \|\nabla f\| \cos\theta$$

- It is largest when $\theta = 0$, that is, when you walk along the gradient. The rate of climb there is the gradient's length, $\|\nabla f\|$.
- It is zero when you walk at right angles to the gradient, along a contour line where the height doesn't change. That is why the arrows in the figure cross the contours at right angles.
- It is most negative when you walk exactly against the gradient. $-\nabla f$ **is the direction of steepest descent.**

In the worked example, $\|\nabla f\| = \sqrt{2^2 + 8^2} = \sqrt{68} \approx 8.25$. No direction climbs faster: north managed 8, and the $(3, 4)$ direction 7.6.

Gradient descent

To make a loss smaller, step against its gradient:

$$\mathbf{w} \;\leftarrow\; \mathbf{w} - \eta\, \nabla L(\mathbf{w})$$

Every weight moves a little, each in proportion to its own partial derivative. The learning rate η sets the size of the step, and Part 6 showed why it can't be too large.

Back to the neuron from Part 7. At $w = 0.5, b = 0$ its loss was 4 and its gradient was $(\partial L/\partial w, \partial L/\partial b) = (-16, -8)$. Take one step with $\eta = 0.01$:

$$w = 0.5 - 0.01 \cdot (-16) = 0.66$$
$$b = 0 - 0.01 \cdot (-8) = 0.08$$

Run the forward pass again: $u = 0.66 \cdot 2 + 0.08 = 1.40$, then $v = 1.96$, then $L = (1.96 - 3)^2 \approx 1.08$. One step cut the loss from 4 to about 1.08.

That is one step of training, and every neural network trains with steps of this shape: a forward pass to get the loss, a backward pass to get the gradient, and a small move downhill. Real training estimates the gradient from a random batch of examples rather than the whole dataset, which is *stochastic* gradient descent, and optimisers such as Adam adjust the step for each weight separately. But all of them start from the gradient. In a large model the gradient has billions of entries, and it is still just a list of partial derivatives.

> **HIDDEN MEANING**
> The gradient is the only direction the loss cares about right now. A step along a contour changes nothing, so a gradient step spends all of its length where it counts. It is also local and greedy: it knows which way is down from *here*, and nothing about where the deepest valley is. Everything a network learns is the accumulated result of these short-sighted steps, each one decided by the training examples voting together.

Try it

1. Find ∇z for $z = x^2 + 4y^2$.
2. For $z = 3x^2 + 2xy + y^2$, evaluate ∇z at $(1, 2)$.
3. With $\nabla f = (2, 8)$, find the directional derivative in the direction $(0, 1)$.
4. With $\nabla f = (3, 4)$, what is the fastest possible rate of climb, and in which direction?

> **▼ ANSWERS**
>
> 1. $(2x, 8y)$
> 2. $(10, 6)$
> 3. 8
> 4. $\|(3, 4)\| = 5$, in the direction $(0.6, 0.8)$, along the gradient itself.

Part 9: Jacobians and Hessians

The Jacobian is the derivative of a function with many outputs, which is what a network layer is. The Hessian is the second derivative of a function with many inputs, which is what shows the shape of the loss.

When the output is a list too

The gradient handles many inputs and one output. A layer of a network has many inputs *and* many outputs. Take a function of x and y that produces two outputs:

$$u = x^2 + y \qquad v = xy$$

There are now four questions: how each output changes with each input. The **Jacobian** arranges the four answers in a grid, with one row per output and one column per input:

$$J = \begin{bmatrix} \dfrac{\partial u}{\partial x} & \dfrac{\partial u}{\partial y} \\ \dfrac{\partial v}{\partial x} & \dfrac{\partial v}{\partial y} \end{bmatrix} = \begin{bmatrix} 2x & 1 \\ y & x \end{bmatrix}$$

Each row is the gradient of one output. Each entry says: nudge this input, and this output moves at this rate.

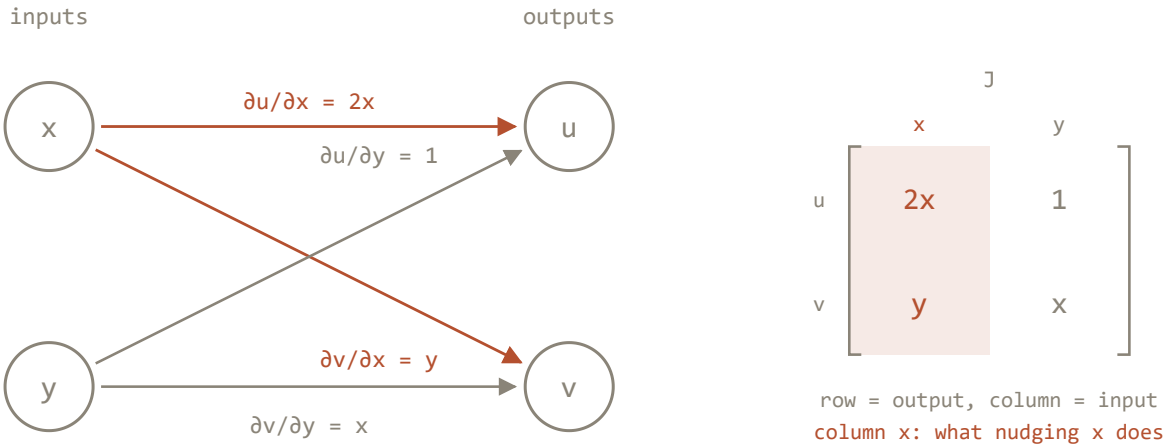


Figure 1. Every input can move every output. The Jacobian lists the four rates: one row per output, one column per input. The first column is everything that nudging x does.

The Jacobian does the job the ordinary derivative did in Part 1, where $\Delta f \approx f'(x)\,\Delta x$. Now it is a matrix times a vector:

$$\begin{bmatrix} \Delta u \\ \Delta v \end{bmatrix} \approx J \begin{bmatrix} \Delta x \\ \Delta y \end{bmatrix}$$

At the point $(2, 3)$ the Jacobian is $\begin{bmatrix} 4 & 1 \\ 3 & 2 \end{bmatrix}$. Nudge x by 0.01, and it predicts $\Delta u \approx 0.04$ and $\Delta v \approx 0.03$. Directly: u goes from 7 to $2.01^2 + 3 = 7.0401$, and v goes from 6 to $2.01 \cdot 3 = 6.03$.

For a linear function the Jacobian is simply its table of coefficients. For $u = 3x + y$ and $v = x + 2y$:

$$J = \begin{bmatrix} 3 & 1 \\ 1 & 2 \end{bmatrix}$$

Jacobians are how gradients pass through layers

A layer of a neural network takes a vector and returns a vector, so its derivative is a Jacobian. For a linear layer $\mathbf{z} = W\mathbf{x}$, the Jacobian is the weight matrix W itself, for the same reason as in the example just above.

Layers are chained, and the chain rule for vectors is the one from Part 3, with matrices in place of numbers:

$$J_{\text{network}} = J_3 \, J_2 \, J_1$$

Backpropagation never builds this whole product. It starts with the gradient of the loss and multiplies it by one layer's Jacobian (transposed) at a time, from the last layer back to the first. The vanishing and exploding gradients of Part 3 are this product of matrices shrinking toward zero or growing without limit.

The Hessian: curvature in every direction

For a function with many inputs and one output, such as a loss, the second derivative is a matrix too. The **Hessian** collects every second partial derivative:

$$H = \begin{bmatrix} \dfrac{\partial^2 f}{\partial x^2} & \dfrac{\partial^2 f}{\partial x \, \partial y} \\ \dfrac{\partial^2 f}{\partial y \, \partial x} & \dfrac{\partial^2 f}{\partial y^2} \end{bmatrix}$$

Take $f(x, y) = x^2 + 3xy + y^2$. Its first partial derivatives are $\partial f/\partial x = 2x + 3y$ and $\partial f/\partial y = 3x + 2y$. Differentiate each of those again, once with respect to x and once with respect to y:

$$H = \begin{bmatrix} 2 & 3 \\ 3 & 2 \end{bmatrix}$$

Two things to notice. First, the off-diagonal entries are equal. For smooth functions they always are, because the order in which you take partial derivatives doesn't matter, so a Hessian is symmetric. Second, the Hessian is the Jacobian of the gradient: the gradient is itself a function with several outputs, and its Jacobian is H.

What the Hessian sees that the gradient can't

At $(0, 0)$, the gradient of $f = x^2 + 3xy + y^2$ is $(0, 0)$, so the surface is flat there. Is it a minimum? Walk along two lines through the origin:

- along $y = x$: $f = x^2 + 3x^2 + x^2 = 5x^2$, which rises in both directions;
- along $y = -x$: $f = x^2 - 3x^2 + x^2 = -x^2$, which falls in both directions.

So the origin is a **saddle**: flat, but uphill one way and downhill another, like the middle of a horse's saddle. The gradient can't tell a saddle from a minimum, since it is zero at both. The Hessian can, because it holds the curvature in every direction. Along a direction (a, b) the curvature is

$$\frac{\partial^2 f}{\partial x^2}\, a^2 + 2\,\frac{\partial^2 f}{\partial x\, \partial y}\, ab + \frac{\partial^2 f}{\partial y^2}\, b^2$$

For $(1, 1)$ that is $2 + 6 + 2 = 10$, and for $(1, -1)$ it is $2 - 6 + 2 = -2$: exactly the second derivatives of $5x^2$ and $-x^2$. Compare $x^2 + y^2$, whose Hessian is $\begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}$. It curves upward in every direction: a true bowl.

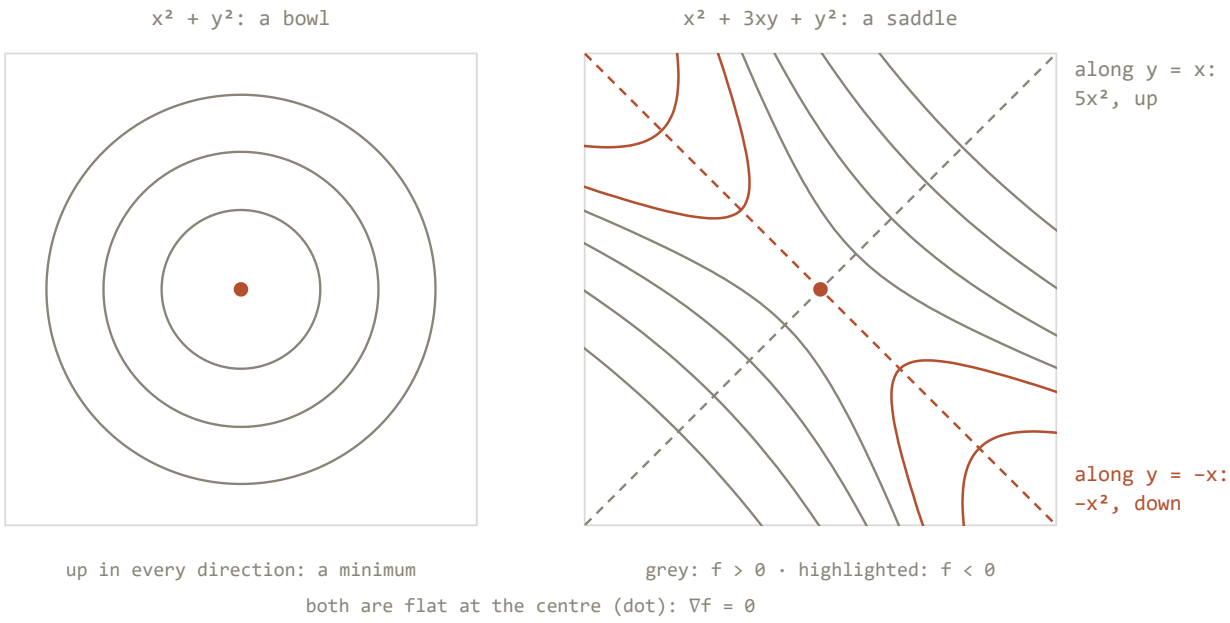


Figure 2. A bowl and a saddle seen from above. Both are flat at the centre, but the saddle rises along y = x and falls along y = −x.

Where this shows up in AI

Saddle points. A large network's loss is a surface over billions of dimensions. With so many directions, it is unlikely that every one of them curves upward at a flat point, and research on high-dimensional problems suggests most flat points are saddles rather than minima.[1] Gradient descent slows down near a saddle, where the gradient is small, but the downhill directions usually let it escape.

Step sizes. Part 6 showed that curvature caps the learning rate at $2/L''$. With many weights, the Hessian holds the curvature in every direction, and the most sharply curved one sets the cap.

Newton's method. If you know the curvature, you can jump straight to the bottom of a bowl. In one dimension, Newton's step divides the slope by the curvature, $w \leftarrow w - L'(w)/L''(w)$. On the one-weight loss $L = (2w - 6)^2$ from the topic page, $L' = 8w - 24$ and $L'' = 8$, so from any starting w a single Newton step lands exactly on $w = 3$, where gradient descent needed many. Why doesn't everyone use it? A model with a billion weights has a Hessian with a billion times a billion, 10^{18}, entries. So training uses the gradient alone, and curvature is approximated cheaply or studied, rather than computed in full.

That takes the series from the slope of a single curve to the curvature of a whole landscape.

> HIDDEN MEANING
> A layer's Jacobian describes what the layer does to small changes: which directions of its input it stretches, which it squashes, and which it ignores. A layer that squashes some directions on the way forward also shrinks the learning signal on the way back, because backpropagation multiplies by the same matrix transposed, and a transposed matrix stretches and squashes by the same amounts. So one object decides both what information survives the forward pass and what learning signal survives the backward one. The Hessian then says what kind of place training has reached: a bowl, a saddle, a narrow ravine or a broad plain.

Try it

1. The Jacobian of $u = 3x + y, v = x + 2y$.
2. The Jacobian of $u = x^2 + y, v = xy + y^2$.
3. The Jacobian of $u = x^2 + 3y, v = 2xy + y^2$.
4. The Hessian of $f(x, y) = x^2 + 4y^2$. What does it say about the contours in the figure in Part 8?

▼ ANSWERS

1. $\begin{bmatrix} 3 & 1 \\ 1 & 2 \end{bmatrix}$
2. $\begin{bmatrix} 2x & 1 \\ y & x + 2y \end{bmatrix}$. The easy slip is the last entry: differentiating xy with respect to y leaves x, so it is $x + 2y$, not $2y$.
3. $\begin{bmatrix} 2x & 3 \\ 2y & 2x + 2y \end{bmatrix}$
4. $\begin{bmatrix} 2 & 0 \\ 0 & 8 \end{bmatrix}$. It curves upward in every direction, so the origin is a minimum, and it curves four times as sharply in y as in x. That is why the contours are squeezed in the y direction.

1. Dauphin et al. (2014), “Identifying and attacking the saddle point problem in high-dimensional non-convex optimization”, *NeurIPS*. ↩

The hidden meaning of the calculus

Anyone can learn to compute a partial derivative. What matters for AI is what each result says about how a network learns. Here is every idea in the series, read that way.

Computing a partial derivative is a skill anyone can pick up in an afternoon. Knowing what it *means* for a network is different, and it is the part that matters. It is what lets you look at an equation in a paper and see a behaviour, a failure, or the idea for a fix. Mechanics make you someone who can do calculus. Meaning makes you someone who can think about AI with it.

The nine parts of this series each end with a *hidden meaning*. Here they are together, because they turn out to tell one story: how a network learns, and where that learning breaks.

A derivative is blame

When a network gets something wrong, every weight shares in the error, but not equally. $\partial L/\partial w$ is how much of it belongs to w, and in which direction. Training is nothing more than measuring that blame for every weight and nudging each one to reduce it.

The blame is only valid nearby. A derivative treats the loss as a straight line around the current weights, which is true for a short distance and false further away. That is why no network is trained in one jump: it is trained by thousands of small corrections, each trusting the straight line for one short step. (Part 1)

Learning is a vote

The loss on a dataset is the sum of the losses on its examples, and derivatives of sums are sums of derivatives. So every example votes on how every weight should move, and the gradient is the tally. A random mini-batch is a fair sample of the vote, which is the only reason training on billions of examples is possible: you never need to count every ballot to know which way the election is going. (Part 2)

Depth is multiplication

A weight in the first layer never touches the loss. Its influence passes through every later layer, and the chain rule says its blame is the *product* of every local rate along the way. That is credit assignment, the central problem of training deep networks.

Products are fragile. One link close to zero silences everything upstream, and a long chain of rates below 1 shrinks to nothing. In a recurrent network the chain runs through time, so vanishing gradients are a statement about memory: an input from 100 steps ago can only shape learning if its influence survives 100 multiplications. Residual connections, LSTM gates and careful initialisation are all ways of keeping that product near 1. (Part 3)

Gates decide what gets learned

The product rule says each factor's change is scaled by the other factor's size. So when a gate multiplies a signal and the gate is near zero, the signal gets almost no gradient: the network has chosen not to learn from it for now. A ReLU does the same thing as a switch. Units that are off pass no gradient back, so each input trains only the part of the network it switched on. (Part 2, Part 4)

A sigmoid that is far from zero has the opposite problem. It is so certain that its slope is almost flat, and a flat slope passes back almost nothing. A saturated unit has stopped listening.

Networks think in log-odds and learn from surprise

The exponential turns adding into multiplying. Raise one softmax score by 1 and its odds against every other option are multiplied by e, so the scores inside a classifier or a language model are really log-odds. The logarithm turns multiplying back into adding, so evidence that multiplies as probability adds as log-probability, and the cross-entropy loss $-\ln p$ is a measure of surprise.

The two meet in the learning signal of the output layer, $p_i - y_i$: what the model believed minus what was true. A confident, correct prediction produces almost no gradient. A confident, wrong one produces a large one. The network learns most from the examples that surprise it. (Part 4)

"All else equal" only holds up close

A partial derivative judges each weight as if it acted alone. That is a useful fiction: in a real step every weight moves together, and their effects interact. The fiction holds for small steps and breaks for large ones, which is another reason learning rates are small.

Look at what the gradient of one neuron says, too: the update to a weight is the signal coming in times the error coming back. That has the shape of Hebb's rule, in which a connection strengthens when the cells on both sides are active together. Here the error coming back stands in for the second cell. (Part 7)

The gradient is short-sighted

The gradient points the way the loss increases fastest from where you stand, and nowhere else. A step along a contour changes nothing, so a gradient step spends all of its length where it counts. But it is greedy and local. It knows which way is down from *here*, and nothing about where the deepest valley is. Every trained network is the accumulated result of many short-sighted steps. (Part 8)

Curvature is how far to trust the slope

The second derivative says how quickly the slope changes, which is to say how soon the straight-line picture stops being true. High curvature means a short trust distance and a small safe learning rate: η has to stay below $2/L''$, and the sharpest direction among billions sets the limit for all of them. Newton's method uses the

curvature to choose the distance as well as the direction. It is too expensive at scale, and much of optimiser design is about cheaply estimating what plain gradient descent is missing.

Curvature also describes what kind of place training has reached. The bottom of a narrow ravine is sensitive to small changes in the weights; the bottom of a broad plain is not. Whether broad, flat minima generalise better is an influential and still debated idea. (Part 6, Part 9)

A layer decides what survives

A layer's Jacobian says how it responds to small changes in its input: which directions it stretches, which it squashes, which it ignores. Backpropagation multiplies by the same matrix transposed, which stretches and squashes by the same amounts. So a layer that throws information away on the way forward also weakens the learning signal on the way back. One matrix governs both what a network can represent and what it can learn. (Part 9)

Some answers have no history

An equilibrium is defined by a condition, *the state where nothing changes any more*, not by the steps that reached it. Implicit differentiation shows how such an answer moves when its inputs move, without the history. For memory this is a deep point: if recall means settling into a fixed point, as in an associative memory, then the memory can be trained at the fixed point itself, whatever path led there. (Part 5)

The whole series in one table

MATHEMATICS	WHAT YOU COMPUTE	WHAT IT MEANS FOR AI
Derivative	A slope	Blame for the error, valid only nearby
Sum rule	Rates add	Every example votes; a batch samples the vote
Product rule	Each factor scaled by the other	Gates decide what gets learned
Chain rule	Rates multiply	Credit assignment; memory fades through long products
Exponential and log	Growth and its inverse	Scores are log-odds; loss is surprise
Softmax gradient	$p_i - y_i$	The network learns from being wrong
Partial derivative	One input at a time	“All else equal”, true only for small steps
Gradient	Steepest climb	Local and greedy: down from here, nothing more
Second derivative	How the slope changes	How far to trust a step; the learning-rate limit
Jacobian	All the output rates	What survives forward and what learning survives backward
Hessian	All the curvatures	Bowl, saddle, ravine or plain
Implicit differentiation	Slope from a condition	Equilibria, and memories, trained without their history

The algebra in this series is standard, and the same for everyone. Reading it this way is what turns it into a tool for thinking about learning.